

Homework 7: BB84 and Error Correction

Quantum Computing Training 2026

Overview

This homework starts with BB84 and then studies a simplified version of the classical post-processing that happens after the quantum transmission step. After basis sifting, Alice and Bob may have strings that are almost the same, but differ in a few positions. Error correction helps Bob fix his string using limited public parity information.

Bit-string convention

All bit strings are combined using XOR, written $\oplus$. For example, $101100 \oplus 001010 = 100110$. If Alice has x_A and Bob has $x_B = x_A \oplus e$, then the string e has a 1 exactly where Bob's bit differs from Alice's bit. e is the string that is used to track errors.

Problem 1: Implementing BB84 in Qiskit

BB84 uses two bases:

$$Z = \{|0\rangle, |1\rangle\}, \quad X = \{|+\rangle, |-\rangle\}.$$

Alice chooses random bits and random bases. Bob chooses random measurement bases. They keep only the positions where their bases match.

- 1.a.** Complete the encoding table. Use 0 in the X basis for $|+\rangle$, and 1 in the X basis for $|-\rangle$.

bit	basis	state sent
0	Z	_____
1	Z	_____
0	X	_____
1	X	_____

- 1.b.** Write Qiskit functions to prepare one BB84 state and to measure one qubit in the Z or X basis.
- 1.c.** Simulate BB84 for $n = 200$ transmitted qubits. Alice chooses random bits a_i and bases b_i ; Bob chooses random bases c_i . Keep only positions with $b_i = c_i$. Report the number of sifted bits and the sifted-key error rate

$$\text{QBER} = \frac{\#\{\text{sifted positions where Alice and Bob disagree}\}}{\#\{\text{sifted positions}\}}.$$

- 1.d.** Add intercept-resend Eve: Eve chooses a random basis, measures, and resends a BB84 state consistent with her result. Run for $n = 200, 1000, 5000$. What QBER do you observe on the sifted bits?

- 1.e. Derive the expected intercept-resend QBER. Eve uses the wrong basis with probability _____. When Eve uses the wrong basis and Alice and Bob use the same basis, Bob disagrees with Alice with probability _____. Thus the expected sifted-key QBER is _____.

Problem 2: Correcting BB84 Errors with a 3-Bit Repetition Method

Suppose Alice and Bob have sifted strings x_A and $x_B = x_A \oplus e$. The string e is the disagreement pattern: a 1 means Bob's bit is wrong in that position. For example, if $x_A = 101$ and $x_B = 111$, then $e = 010$, so Bob should flip the middle bit to recover 101. The problem is that Bob does not know x_A , so he does not know e directly.

In this problem, work in blocks of three bits. The repetition method treats 000 and 111 as the two clean reference blocks (aka *codewords*). This method can fix one bit flip in a block of three.

A *syndrome* is a short parity-check summary. For a three-bit block $z = z_1 z_2 z_3$, define

$$s(z) = \begin{pmatrix} z_1 \oplus z_2 \\ z_2 \oplus z_3 \end{pmatrix}.$$

This records whether neighboring bits agree or disagree.

- 2.a. Compute $s(e)$ for $e = 000, 100, 010, 001$. Make a lookup table: syndrome $\rightarrow$ error to flip.
- 2.b. Alice publicly sends $s(x_A)$. Bob computes $s(x_B)$. Show that $s(x_B) \oplus s(x_A) = s(e)$. Why does this let Bob identify a single-bit error without Alice revealing the whole block?
- 2.c. Work through the example $x_A = 101$, $x_B = 111$. Compute $s(x_A)$, $s(x_B)$, their XOR, the inferred error, and Bob's corrected block.
- 2.d. Implement this three-bit reconciliation method in Python. Split Alice's and Bob's sifted keys into three-bit blocks, compute the syndrome difference, use your lookup table, and correct Bob's block. Ignore leftover bits.
- 2.e. Test on synthetic keys of length $m = 3000$. Let Alice's key be random, and flip each of Bob's bits with probability $q \in \{0.01, 0.03, 0.05, 0.10, 0.15\}$. Report Bob's error rate before and after correction.
- 2.f. Why does the method fail when a block has two or three errors? How many syndrome bits are publicly revealed per three-bit block, and why does public leakage motivate privacy amplification in BB84?

Problem 3: 7-Bit Parity-Check Method

The three-bit method is easy, but it uses many public parity bits. A more efficient classical method uses seven-bit blocks and three parity checks. It can correct one error in each seven-bit block.

Use the parity-check matrix

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

For a seven-bit string z , define its syndrome by $s(z) = Hz \pmod{2}$. If exactly one bit flips in position j , then $s(e)$ is the j -th column of H . This tells Bob which bit to flip.

- 3.a.** Make a decoding table. For each single-bit error position $j = 1, \dots, 7$, compute the corresponding syndrome, i.e. the j -th column of H . Include the all-zero syndrome for “no error.”
- 3.b.** As before, Alice sends $s(x_A)$, Bob computes $s(x_B)$, and Bob uses $s(x_B) \oplus s(x_A) = s(e)$. Work through the example $x_A = 1011010$, $x_B = 1010010$. Which bit should Bob flip?
- 3.c.** Implement seven-bit reconciliation. Split the keys into seven-bit blocks, compute the syndrome difference, use your table to correct Bob’s block, and ignore leftover bits.
- 3.d.** Compare the three-bit and seven-bit methods on synthetic keys of length $m = 7000$, using $q \in \{0.01, 0.03, 0.05, 0.10, 0.15\}$. Report the error rate before correction, after three-bit correction, and after seven-bit correction.
- 3.e.** For the seven-bit method, a block fails if it has two or more errors. Compute

$$p_{\geq 2} = 1 - (1 - q)^7 - 7q(1 - q)^6.$$

Compare this prediction with the failure behavior in your simulation.

References

- Course lecture notebook: [Lecture 7: BB84 Quantum Key Distribution](#).
- Peter Shor, [Lecture 31 notes on BB84, the adapted Lo-Chau protocol, and CSS-code security intuition](#).